



Bitacora de Prompts

Construccion del Sistema Huerto Vivo

Base de Datos · Backend · Frontend

Conversacion con Claude (Anthropic) — claude.ai



3 Prompts

Base de Datos — PostgreSQL



3 Prompts

Backend — Node.js + Express



3 Prompts

Frontend — React + Vite

Introduccion

Este documento evidencia los prompts mas efectivos utilizados en Claude (Anthropic) para construir el sistema Huerto Vivo — una plataforma de gestion comunitaria para 18 vecinos que administran un huerto urbano.

Cada prompt fue seleccionado por su impacto directo en el resultado: no se incluyen prompts de exploracion o conversacion, sino unicamente aquellos que generaron outputs de produccion que se integraron en el sistema final.

La efectividad de un prompt no se mide solo por lo que pide, sino por como aprovecha el contexto acumulado, la precision del lenguaje y la combinacion de elementos visuales y textuales. Estos 9 prompts demuestran que una instruccion bien construida puede generar, en un solo intercambio, lo que normalmente tomaria dias de desarrollo.

Metrica	Base de Datos	Backend	Frontend
Prompts utilizados	3	3	3
Tablas / Rutas / Pantallas generadas	10 tablas + 3 vistas	9 modulos de rutas	9 pantallas funcionales
Errores resueltos en el prompt	1 (tipos ENUM)	1 (SSL local)	1 (cosechas.reduce)
Iteraciones para llegar al resultado	2 versiones	2 versiones	3 iteraciones

1. Base de Datos

Los tres prompts de base de datos construyeron el fundamento de todo el sistema. El primero genero el esquema completo; el segundo lo refactorizo con autenticacion; el tercero resolvió el unico error tecnico de forma inmediata.

#1

Base de
Datos

Generacion del esquema relacional completo

Con los pain points y requerimientos ya definidos, se solicito la base de datos completa en un unico prompt.

Prompt utilizado:

"quiero que me ayudes a generar una base de datos DDL Y DML con postgresSQL"

Por que fue un prompt efectivo

Este prompt fue efectivo por su brevedad calculada: llego despues de una larga fase de analisis donde Claude ya tenia el dominio completo del problema (18 vecinos, pain points, requerimientos). Al no especificar la estructura, Claude pudo interpretar el contexto acumulado y generar el esquema mas apropiado. Un prompt largo habria limitado el resultado a lo que el usuario imaginaba; uno corto permitio que la solucion tecnica surgiera del contexto.

Resultado e impacto directo

Con un unico prompt se genero una base de datos de produccion completa, lista para conectar al backend:

- 7 tablas relacionales con claves foraneas correctas y sin redundancias
- 4 tipos ENUM (estado_turno, estado_parcela, rol_vecino, estado_cosecha) para garantizar integridad de datos
- Constraint unico uq_parcela_activa: impide fisicamente que dos siembras ocupen la misma parcela — resuelve el pain point de duplicacion espacial a nivel de base de datos, sin depender de logica en el backend
- 3 vistas SQL (v_riego_hoy, v_cosechas_mes, v_ranking_vecinos) que responden directamente las preguntas del dashboard
- DML con 18 vecinos de prueba, 18 parcelas en grilla 6x3, 8 especies y datos de turnos de los ultimos 7 dias

#2

Base de
Datos

Refactorizacion total con autenticacion integrada

Despues de iterar con el backend, se detecto que la BD necesitaba soporte de autenticacion real (login con email y contraseña) y tablas adicionales para chat y notificaciones.

Prompt utilizado:

"quiero que empecemos desde la base de datos, haciendola nuevamente con todo lo necesario, desde el inicio de sesion, hasta todo lo que llevamos"

Por que fue un prompt efectivo

Pedir una refactorizacion total en el momento preciso — cuando el proyecto ya habia madurado — fue mas eficiente que hacer cambios incrementales. La frase 'desde el inicio de sesion' actuó como ancla

semantica: le indico a Claude que la autenticacion debia estar integrada en el diseno, no agregada como parche. 'Todo lo que llevamos' transfiro todo el contexto acumulado sin necesidad de repetirlo.

Resultado e impacto directo

La version 2.0 de la BD fue un salto cualitativo que habilito el sistema de login real y el chat persistente:

- Columna password_hash en tabla vecinos con hashes bcrypt pregenerados para los 18 vecinos (contraseña '1234') — autenticacion lista para usar sin configuracion adicional
- 2 tablas nuevas: mensajes_chat (reemplaza el WhatsApp caótico) y notificaciones (avisos tipados por categoria)
- Tabla actividad_vecinos para calcular el ranking mensual de compriso — base del dashboard de rendicion de cuentas
- Indices de optimizacion en columnas de busqueda frecuente (fecha_turno, vecino_id, estado)
- Script de verificacion al final: consulta que muestra el conteo de cada tabla para confirmar que los datos se insertaron correctamente

#3

Base de
Datos

Correccion de error de tipos ENUM

Al ejecutar el script en pgAdmin, aparecio un error de tipo en el INSERT de turnos_riego. Se reporto con imagen.

Prompt utilizado:

"[imagen del error en pgAdmin] column 'estado' is of type estado_turno but expression is of type text - LINE 353"

Por que fue un prompt efectivo

Reportar el error con una captura de pantalla en lugar de describirlo en texto fue determinante. La imagen mostro la linea exacta (353), el nombre de la columna y el tipo esperado — informacion que habria tomado varios intercambios obtener por texto. Este tipo de prompt visual es especialmente efectivo para errores tecnicos donde el contexto visual (color del error, numero de linea, mensaje exacto) contiene mas informacion que su descripcion.

Resultado e impacto directo

El error se resolvio en un unico intercambio con un fix que previno recurrencias futuras:

- Causa identificada: PostgreSQL no convierte automaticamente texto a ENUM en bloques VALUES, requiere cast explicito
- Fix aplicado: agregar ::estado_turno en el INSERT — solucion puntual de una linea
- Fix preventivo: se revisaron y corrigieron todos los demas ENUMs del script (::estado_parcela, ::rol_vecino, ::estado_cosecha) antes de que causaran el mismo error
- Tiempo de resolucion: un solo intercambio sin iteraciones adicionales

2. Backend

El backend se construyo en dos iteraciones creativas y una correccion tecnica. La primera genero el servidor completo; la segunda agrego las rutas avanzadas del coordinador; el error SSL se resolvió en un intercambio.

#4

Backend

Servidor REST completo en un prompt

Con la base de datos lista y el stack tecnologico definido, se solicito el backend completo.

Prompt utilizado:

"ahora necesito el backend, y con esto dejarlo conectado a la base de datos para que operen en conjunto"

Por que fue un prompt efectivo

La efectividad de este prompt esta en dos palabras: 'en conjunto'. No se pidio solo un servidor, sino un servidor que opere en conjunto con la base de datos ya construida. Eso le indico a Claude que el resultado debia incluir la capa de conexion, la autenticacion contra la tabla vecinos, y las rutas coherentes con el esquema de la BD. El adverbio 'ahora' establecio continuidad con el trabajo anterior sin necesidad de repetir el contexto.

Resultado e impacto directo

Se genero un backend de produccion completo con 9 modulos de rutas, listo para conectar al frontend:

- 9 archivos de rutas independientes: auth, vecinos, parcelas, siembras, turnos, cosechas, dashboard, chat, notificaciones
- Middleware JWT con control de roles: soloCoordinador bloquea rutas sensibles para miembros regulares
- Validacion Zod en todos los endpoints POST y PATCH — rechaza datos malformados antes de tocar la BD
- Transacciones en operaciones criticas: registrar una siembra bloquea la parcela atomicamente; reportar cosecha actualiza las metricas en la misma transaccion
- Health check en GET /health que prueba la conexion real a PostgreSQL — no simula el estado

#5

Backend

Diagnostico y correccion de error SSL

Al ejecutar el backend, el health check devolvio db_error. Se agrego codigo de diagnostico y se reporto el error real.

Prompt utilizado:

"[imagen del navegador] The server does not support SSL connections"

Por que fue un prompt efectivo

Este prompt fue efectivo por la misma razon que el prompt #3: una imagen del error exacto vale mas que mil palabras. El mensaje 'The server does not support SSL connections' es un error conocido y especifico de PostgreSQL local que tiene una causa y solucion bien definidas. Al verlo directamente, Claude pudo identificar inmediatamente que el problema era la configuracion SSL del pool de conexiones, no la cadena de conexion ni las credenciales.

Resultado e impacto directo

La correccion resolvió el problema de conexión para todos los entornos posibles con una sola modificación:

- Causa identificada: el código configuraba `ssl: { rejectUnauthorized: false }` que es correcto para Supabase pero falla en PostgreSQL local
- Fix elegante: lógica automática con `const isLocal = DATABASE_URL.includes('localhost')` — detecta el entorno sin variables adicionales
- Resultado: `ssl: false` para PostgreSQL local, `ssl: { rejectUnauthorized: false }` para Supabase u otros servicios cloud
- Beneficio adicional: el mismo código funciona en desarrollo local y en producción cloud sin cambiar el `.env`

#6

Backend

Rutas avanzadas de rotación y mapa

Con el backend base funcionando, se necesitaron dos módulos adicionales para las funcionalidades exclusivas del coordinador.

Prompt utilizado:

"agrega estas dos: Recordatorio por turno — al asignar un turno se genera automáticamente una notificación para el vecino responsable. Rotación automática de turnos — el coordinador define un patrón y el sistema genera los turnos del mes entero automáticamente. En las de media prioridad agrega todas, y en baja prioridad agrega el modo offline que es demasiado útil"

Por que fue un prompt efectivo

La estructura de este prompt fue inusualmente eficiente: en lugar de describir las rutas técnicamente, describió el comportamiento esperado desde la perspectiva del usuario ('el coordinador define un patrón', 'se genera automáticamente una notificación'). Eso le permitió a Claude inferir la arquitectura correcta de rutas, la lógica de negocio y los endpoints necesarios. La frase 'es demasiado útil' funcionó como señal de prioridad que elevó el modo offline de 'baja prioridad' a implementación real.

Resultado e impacto directo

6 funcionalidades nuevas implementadas en una sola iteración:

- Ruta POST `/api/rotacion/generar`: distribuye turnos entre vecinos activos con lógica de rotación configurable (intervalo 1-7 días)
- Ruta GET `/api/rotacion/preview`: genera la vista previa sin guardar en BD — permite revisar antes de confirmar
- Ruta POST `/api/rotacion/recordatorio`: crea notificaciones para todos los vecinos con turno al día siguiente
- Ruta GET `/api/mapa`: devuelve la grilla 6x3 con estado de riego de cada parcela en tiempo real
- Modo offline en frontend: cola `localStorage` + sincronización automática al recuperar conexión
- Navegación diferenciada por rol: coordinador ve Mapa y Rotación; miembro ve Siembras y Cosecha

3. Frontend

El frontend partio de una imagen de prototipo y se construyo en tres iteraciones: la primera genero la estructura visual; la segunda conecto cada pantalla al backend real; la tercera resolvió bugs y completo la integracion.

#7

Frontend

App completa desde imagen de prototipo

Con backend y base de datos operativos, se solicito el frontend usando como referencia una captura del prototipo creado en Google Stitch.

Prompt utilizado:

"[imagen del prototipo Stitch con 5 pantallas] ya tengo listo la base de datos y el backend, ayudame a generar el front end inspirado en este prototipo de stitch"

Por que fue un prompt efectivo

Este fue el prompt mas poderoso del proyecto por la combinacion de dos elementos: el contexto tecnico acumulado ('ya tengo listo la base de datos y el backend') y la referencia visual concreta (imagen del prototipo). El contexto tecnico le indico a Claude que el frontend debia conectarse a endpoints reales, no usar datos mock. La imagen aportó la paleta de colores, la estructura de navegacion y el estilo visual sin necesidad de especificarlos en texto. La palabra 'inspirado' fue clave — indico adaptacion creativa, no copia exacta.

Resultado e impacto directo

Se genero una aplicacion React mobile-first completa, conectada al backend, con diseno coherente:

- 5 pantallas funcionales: Inicio (dashboard), Siembras, Riego, Cosecha y Chat — cada una conectada a sus endpoints correspondientes
- api.js centralizado con 22 funciones que cubren todos los endpoints del backend
- Paleta verde comunitaria extraida automaticamente del prototipo: --verde-oscuro, --verde-medio, --verde-suave, --verde-mint
- Navegacion inferior con tabs, modales con animacion slideUp, tarjetas con sombras y bordes redondeados
- Modo demo automatico: si el backend no responde, la app usa datos mock y sigue siendo funcional

#8

Frontend

Integracion real con backend y funciones clave

El frontend inicial tenia las pantallas pero no estaba completamente conectado al backend. Se necesitaba que las acciones principales guardaran datos reales.

Prompt utilizado:

"quiero que de el front end, que ademas de ya estar entrelazado con el backend y el sql, agregues: activar campana para informar sobre asuntos importantes, agregar desglose para reportar cosecha, arreglar orden de los mas activos segun los datos de la base de datos, cuando marques el turno este quede en la base de datos, el chat quede registrado"

Por que fue un prompt efectivo

La estructura de lista con verbos imperativos especificos fue determinante. Cada item tenia un verbo de accion claro: activar (campana), agregar (desglose), arreglar (orden), quede registrado (turno y chat). Esta estructura le permitio a Claude tratar cada requerimiento de forma independiente y verificar que cada uno quedara implementado. A diferencia de una descripcion en prosa, la lista impidio que algun requerimiento quedara sin abordar.

Resultado e impacto directo

La app paso de ser un prototipo visual a un sistema funcional conectado a la BD:

- PATCH /api/turnos/:id/cumplir conectado al boton principal — cada click guarda en PostgreSQL con timestamp exacto
- Chat persistente: POST /api/chat guarda cada mensaje, GET /api/chat recupera historial, autorefresh cada 8 segundos
- Ranking ordenado por pct_cumplimiento DESC calculado en la BD — no en el frontend
- Panel de notificaciones con badge rojo de no leidas, tipos de aviso diferenciados por color (riego/cosecha/alerta/info)
- Formulario de cosecha desglosado: selector de siembra activa, calidad percibida, cantidad en kg, fecha y notas opcionales

#9

Frontend

Correccion de bug con verificacion ampliada

Al abrir la pantalla de Cosecha, la app se rompia con un error de JavaScript. Se reporto con screenshot y se aprovecho para pedir revision general.

Prompt utilizado:

"[imagen del error en consola] cosechas.reduce is not a function — sale este error al colocar las cosechas, ademas de que no incluye el cerrar sesion, solucionalo y verifica si existen algunas otras incongruencias"

Por que fue un prompt efectivo

La efectividad de este prompt viene de tres elementos combinados. Primero, la imagen del error mostro el mensaje exacto y el stack trace sin necesidad de transcribirlo. Segundo, agregar un segundo problema ('no incluye el cerrar sesion') en el mismo prompt evito un intercambio adicional. Tercero, y mas importante, la frase 'verifica si existen otras incongruencias' transformo un prompt de correccion de un bug en una revision general — sin ese agregado, solo se habria corregido el error reportado.

Resultado e impacto directo

Se corrigieron 4 problemas en una sola iteracion, incluyendo 3 no reportados:

- Bug principal resuelto: el backend devuelve { ok, data: { detalle: [] } } pero el codigo intentaba .reduce() sobre el objeto completo — fix con `Array.isArray(resp?.data?.detalle) ? resp.data.detalle : []`
- Boton de cerrar sesion agregado en barra superior permanente con nombre del vecino y rol
- El mismo patron de acceso incorrecto a datos detectado y corregido en Siembras.jsx e Inicio.jsx (ranking) — bugs que habrian aparecido al abrir esas pantallas
- Normalizacion unificada de respuestas del backend en todos los componentes: `Array.isArray(resp?.data) || Array.isArray(resp?.data?.detalle)`

Patrones de Efectividad

El analisis de los 9 prompts revela patrones claros que explican por que fueron efectivos:

Patron	Como se aplico en este proyecto
Contexto acumulativo	Los prompts mas cortos funcionaron mejor. 'Ahora necesito el backend' genero 9 modulos de rutas porque Claude tenia el esquema completo de la BD. Un prompt largo habria limitado el output a lo que el usuario imaginaba.
Imagen del error	Los 3 errores tecnicos (ENUM, SSL, cosechas.reduce) se resolvieron en un unico intercambio cada uno. La imagen mostro el mensaje exacto, la linea del error y el contexto — mas informacion que cualquier descripcion en texto.
Verbos de accion especificos	'Activar campana, agregar desglose, arreglar orden, quede registrado' — cada verbo fue una instruccion independiente. La lista con verbos impidio que algun requerimiento quedara sin implementar.
Refactorizacion en el momento justo	Pedir 'rehacerla desde cero con todo lo que llevamos' cuando el proyecto habia madurado genero una BD version 2.0 con autenticacion y tablas nuevas — mas eficiente que cambios incrementales.
Verificacion ampliada	Agregar 'verifica si existen otras incongruencias' a un prompt de correccion de bug resulto en la deteccion de 3 bugs adicionales en otros archivos con el mismo patron erroneo.
Visual + contexto tecnico	La imagen del prototipo Stitch + 'ya tengo listo el backend' genero un frontend coherente, conectado y con paleta de colores correcta — sin especificaciones adicionales de disenio.
Lenguaje de usuario, no tecnico	'El coordinador define un patron y el sistema genera los turnos del mes' — descripciones en lenguaje de usuario permitieron que Claude infiriera la arquitectura tecnica correcta.